

Programação Visual e Autoria Web

Web Storage API

Universidade Federal do Rio Grande do Norte

Web Storage API

- Web Storage API é uma API do HTML5 para armazenar dados no navegador do cliente.
- **Alternativa moderna a cookies, com maior capacidade e sem enviar dados ao servidor em cada requisição.**

Principais características

- Armazenamento de dados de forma simples e segura.
- Dados disponíveis apenas no navegador onde foram armazenados.
- Dividida em dois tipos:
 - **Local Storage:** dados persistem mesmo após fechar o navegador.
 - **Session Storage:** dados são perdidos ao fechar a aba/navegador.

Vantagens do Web Storage

- Capacidade de até 10MB em alguns navegadores, comparado a 4KB dos cookies.
- Fácil de usar com JavaScript.
- Controle do front end: não depende de servidores ou headers HTTP.
- Privacidade melhorada: dados não são enviados automaticamente.

Limitações

- Apenas dados em texto podem ser armazenados.
- Não é ideal para armazenamento de dados sensíveis.

Local Storage vs Session Storage

Característica	Local Storage	Session Storage
Persistência	Dados persistem após fechar o navegador	Dados são removidos ao fechar a aba.
Capacidade	Igual para ambos, cerca de 5-10MB por domínio.	
Uso principal	Dados que precisam ser acessados várias vezes.	Dados temporários para a sessão atual.

- Ambos compartilham o mesmo modelo de segurança:
 - Somente acessíveis pelo mesmo domínio e protocolo.

Local Storage - Métodos e Exemplos

- `localStorage.setItem('key', 'value')`: Armazena dados.
- `localStorage.getItem('key')`: Recupera dados.
- `localStorage.removeItem('key')`: Remove um item.
- `localStorage.clear()`: Remove todos os itens.

Local Storage - Métodos e Exemplos

```
// Armazenando dados
localStorage.setItem('username', 'JohnDoe');

// Recuperando dados
let username = localStorage.getItem('username');
console.log(username); // Output: JohnDoe

// Removendo dados
localStorage.removeItem('username');

// Limpando todo o localStorage
localStorage.clear();
```


Session Storage - Métodos e Exemplos

- `sessionStorage.setItem('key', 'value')`
- `sessionStorage.getItem('key')`
- `sessionStorage.removeItem('key')`
- `sessionStorage.clear()`

Session Storage - Métodos e Exemplos

```
// Armazenando dados
sessionStorage.setItem('sessionId', 'abc123');

// Recuperando dados
let sessionId = sessionStorage.getItem('sessionId');
console.log(sessionId); // Output: abc123

// Removendo dados
sessionStorage.removeItem('sessionId');

// Limpando todos os dados da sessão
sessionStorage.clear();
```

Iterando sobre os Dados Armazenados

- Acessando todas as chaves:
 - `localStorage.length`: Retorna o número de itens armazenados.
 - `localStorage.key(index)`: Recupera o nome da chave em uma posição específica.

```
// Adicionando itens
localStorage.setItem('name', 'Alice');
localStorage.setItem('age', '30');

// Iterando sobre o localStorage
for (let i = 0; i < localStorage.length; i++) {
  let key = localStorage.key(i);
  let value = localStorage.getItem(key);
  console.log(`${key}: ${value}`);
}
// Output: name: Alice, age: 30
```

Boas Práticas

- Use chaves descritivas.
- Limpe dados desnecessários regularmente.
- Prefira usar objetos JSON para estruturas mais complexas.

Boas Práticas

```
// Salvando um objeto como JSON
let user = { id: 1, name: 'Alice' };
localStorage.setItem('user', JSON.stringify(user));

// Recuperando e convertendo o JSON de volta em objeto
let retrievedUser = JSON.parse(localStorage.getItem('user'));
console.log(retrievedUser.name); // Output: Alice
```

Segurança

- Evite armazenar dados sensíveis, como senhas.
- Use HTTPS para proteger dados durante a comunicação.
- Sempre remova dados que não são mais necessários.

Limitações do Web Storage

- Não suporta armazenamento de dados binários.
- Dados são acessíveis por JavaScript, tornando-se vulneráveis se scripts maliciosos acessarem o DOM.
- Não substitui bancos de dados.

Atividade

- Crie uma página simples em HTML com um formulário de login (campos de usuário e senha).
- Utilize o Local Storage para armazenar o nome do usuário.
- Exiba uma mensagem de boas-vindas quando o usuário voltar à página.
- Implemente uma funcionalidade para "Sair", que limpe os dados armazenados.

Referências

- https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function